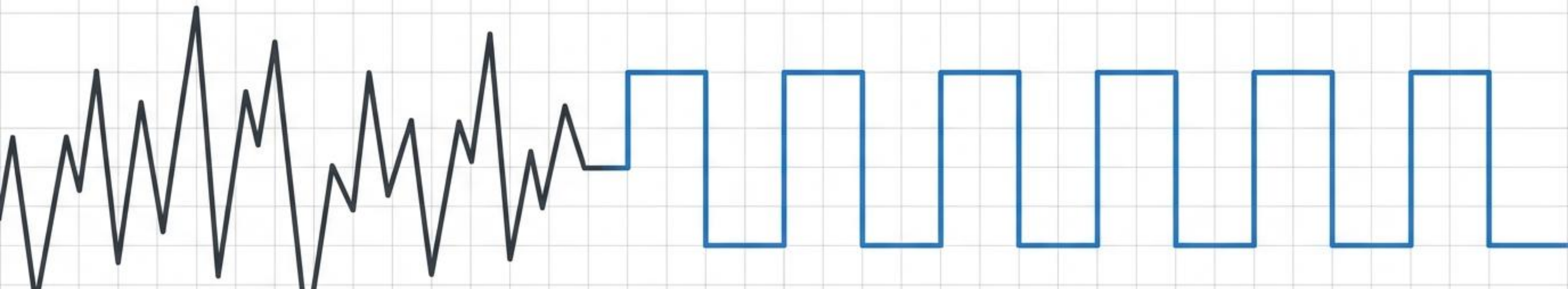


Manajemen Task FreeRTOS

Membangun Arsitektur Multi-tasking Deterministik pada ESP32 dan STM32

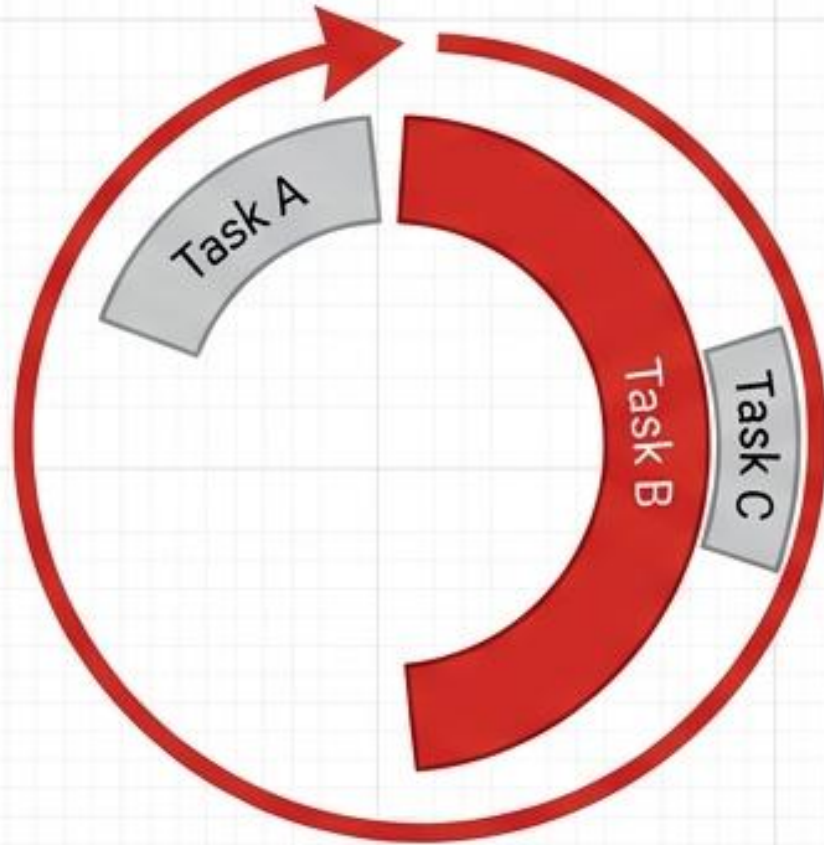


Nama : Anggun Adgita Dwiyuma Lasanov
NIM : 40040324620001
Kelas : Praktikum A
Mata Kuliah : Praktikum Sistem Embedded
Dosen Pengampu : Rofiq Cahyo Prayogo, S.T., M.T.

Modul 08 | Real-Time Operating System

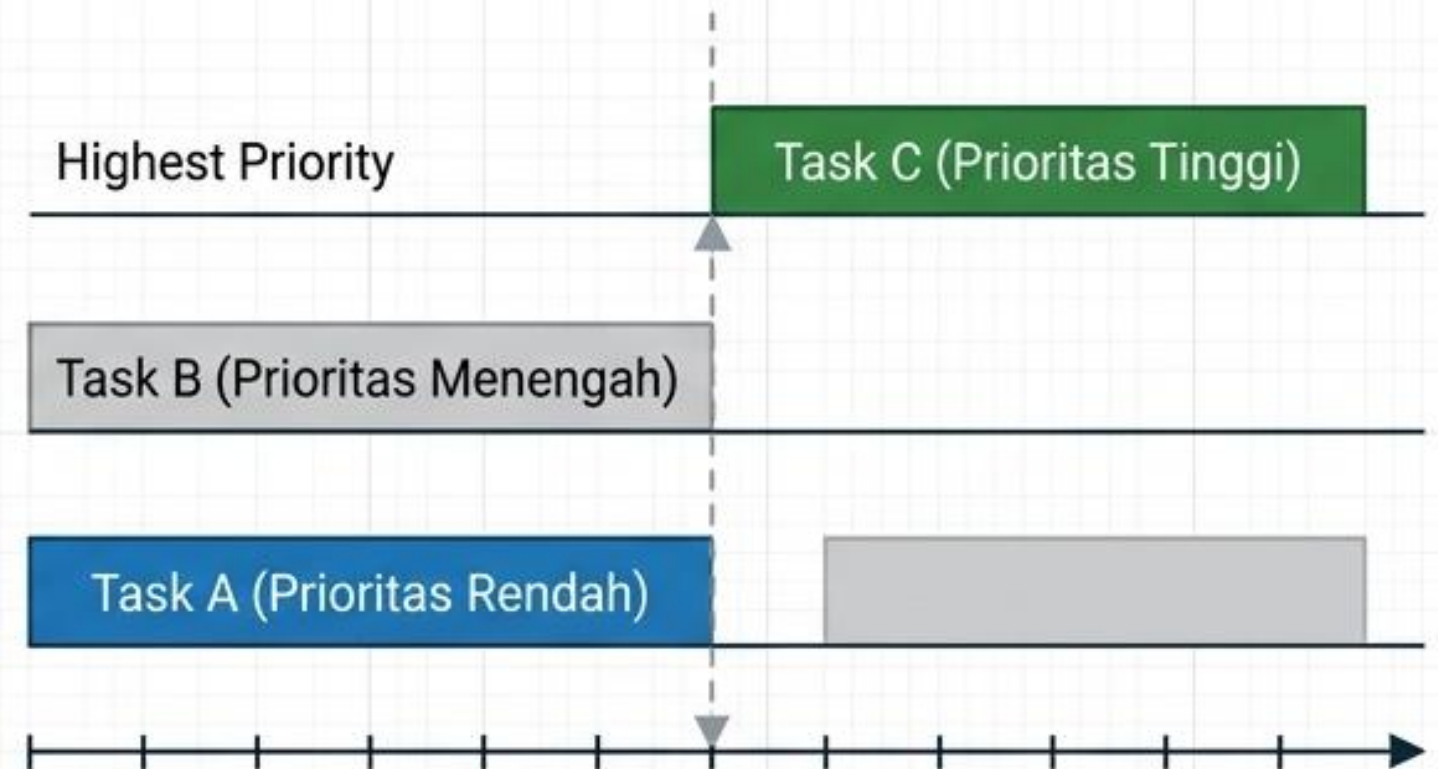
Mengapa Beralih dari Super Loop ke RTOS?

Bare-Metal / Super Loop



Satu loop eksekusi melingkar, rawan bottleneck.

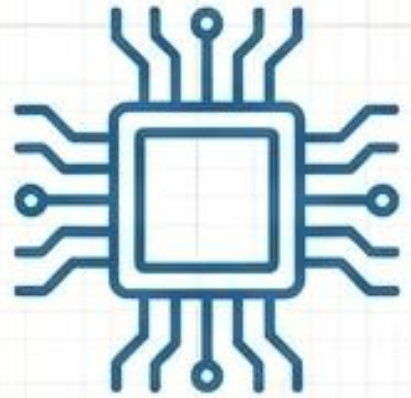
RTOS



Preemptive Scheduling: Paralel dan instan.

RTOS menjamin Deterministic Timing (waktu respons yang dapat diprediksi) dan Low Latency terhadap sebuah event, hal yang mustahil dijamin oleh sistem bare-metal murni saat beban kerja meningkat.

Standar Industri Embedded Systems



Footprint Minimalis

< 10KB ROM & < 1KB RAM

Sangat efisien, ideal untuk mikrokontroler dengan memori terbatas.



Portabilitas Tinggi

35+ Arsitektur

Berjalan di hampir semua platform silikon di industri.



Fitur Sinkronisasi Penuh

**Tasks, Queues,
Mutexes, Timers**

Ekosistem lengkap untuk komunikasi antar-proses yang aman.

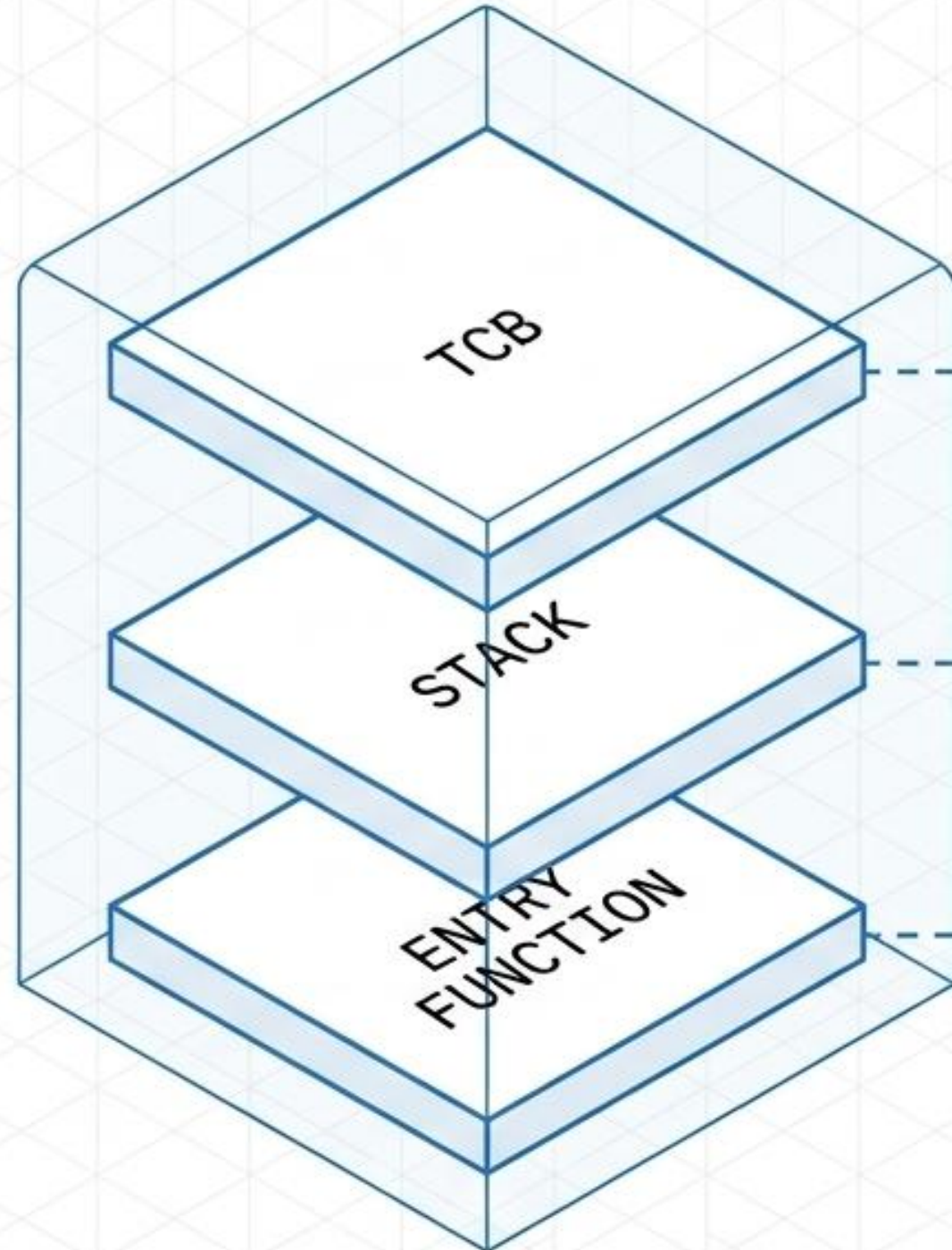


Lisensi Terbuka

MIT License

Terbuka penuh dan gratis untuk implementasi rilis komersial.

Anatomi Fundamental Sebuah Task



1. Task Control Block (TCB)

Metadata vital sistem. Berfungsi sebagai KTP/Paspor milik Task yang menyimpan state saat ini, tingkat prioritas, dan pointer menuju stack.

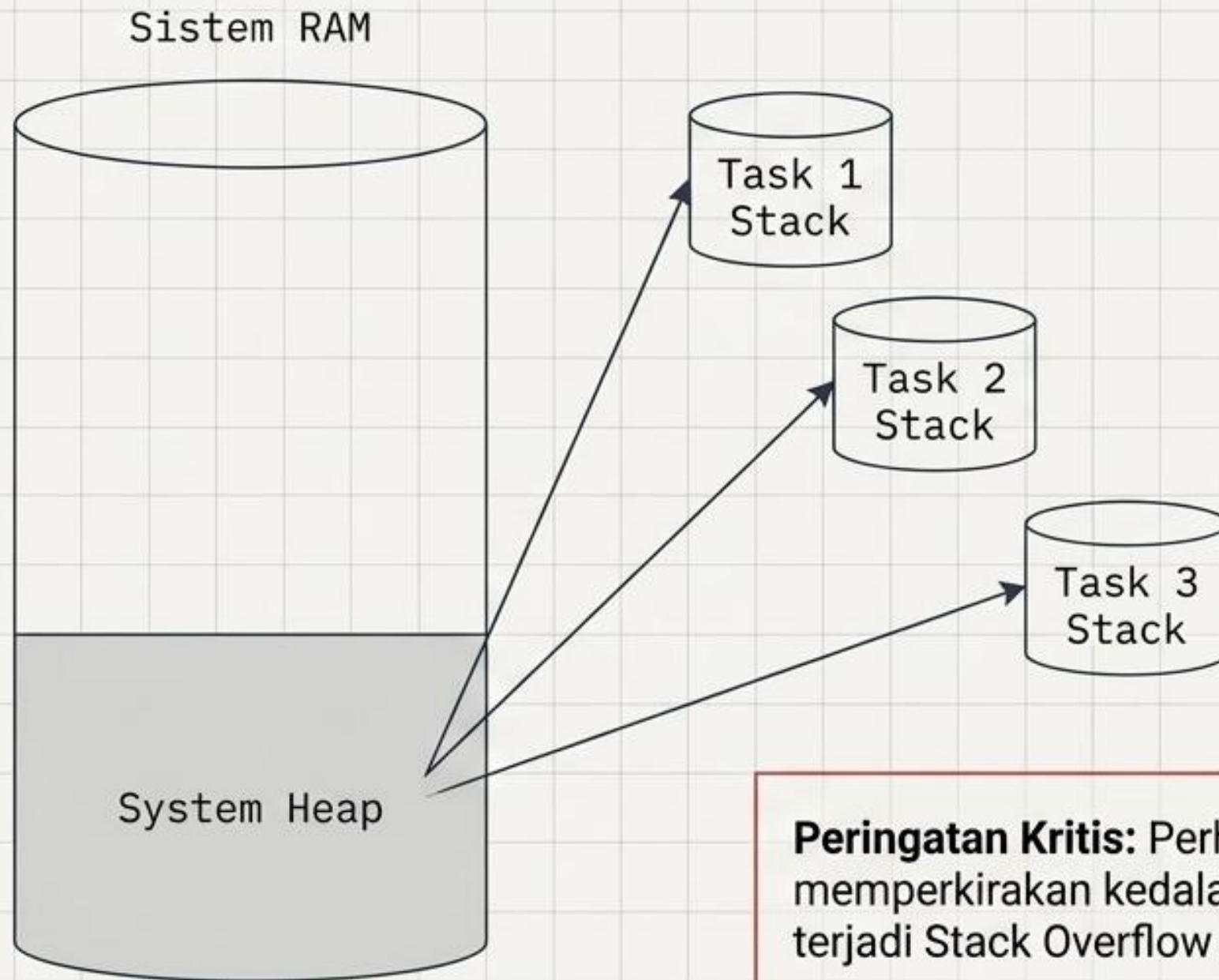
2. Private Stack

Ruang memori terisolasi. Menyimpan variabel lokal dan merekam status register CPU yang spesifik untuk task ini setiap kali terjadi interupsi.

3. Entry Function

Kode C (C code) eksekusi. Sebuah rutinitas infinite loop independen yang melakukan pekerjaan utama dari task tersebut.

Arsitektur Memori: Stack independen dari System Heap

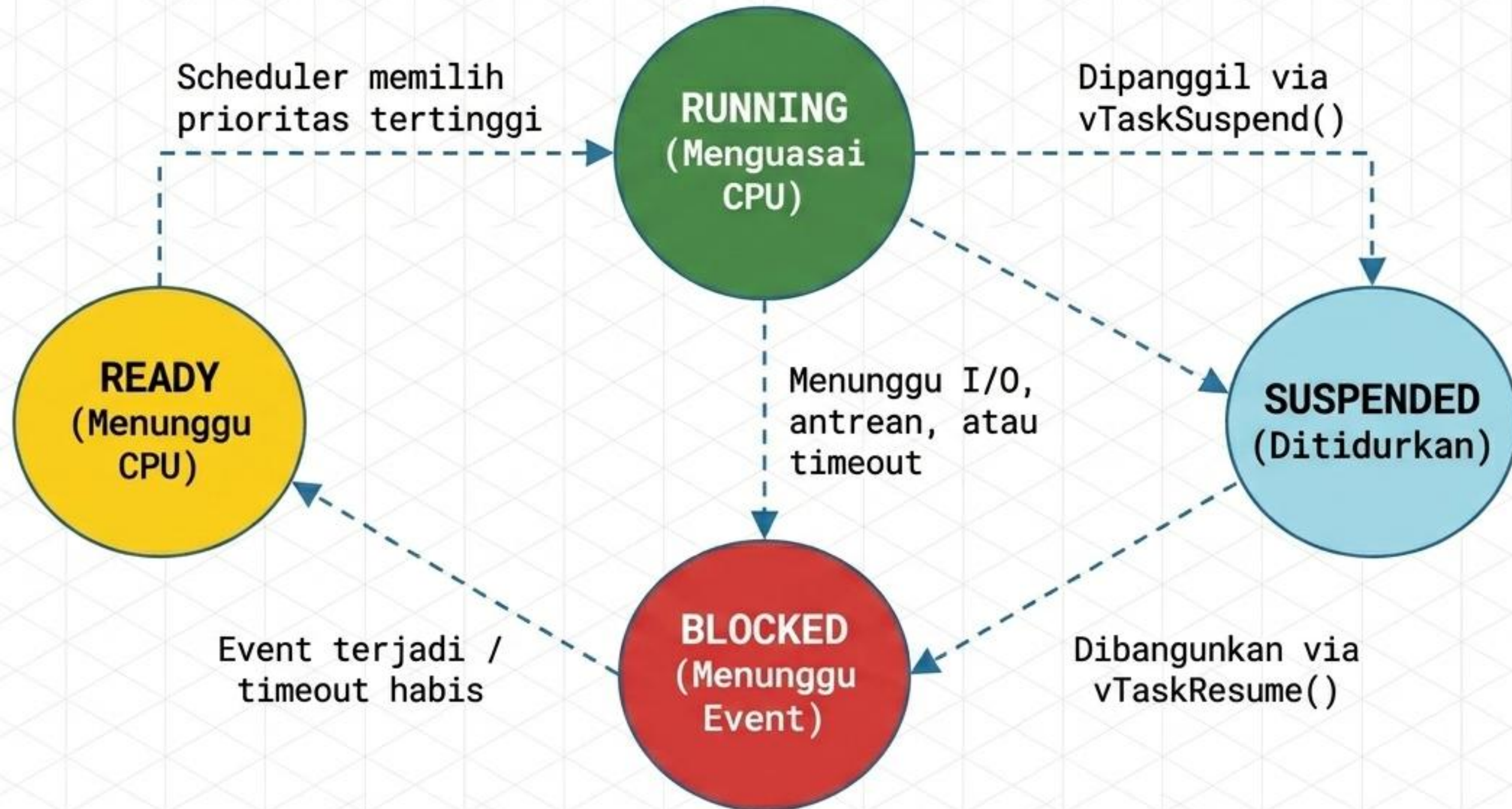


Alokasi Dinamis Terdedikasi

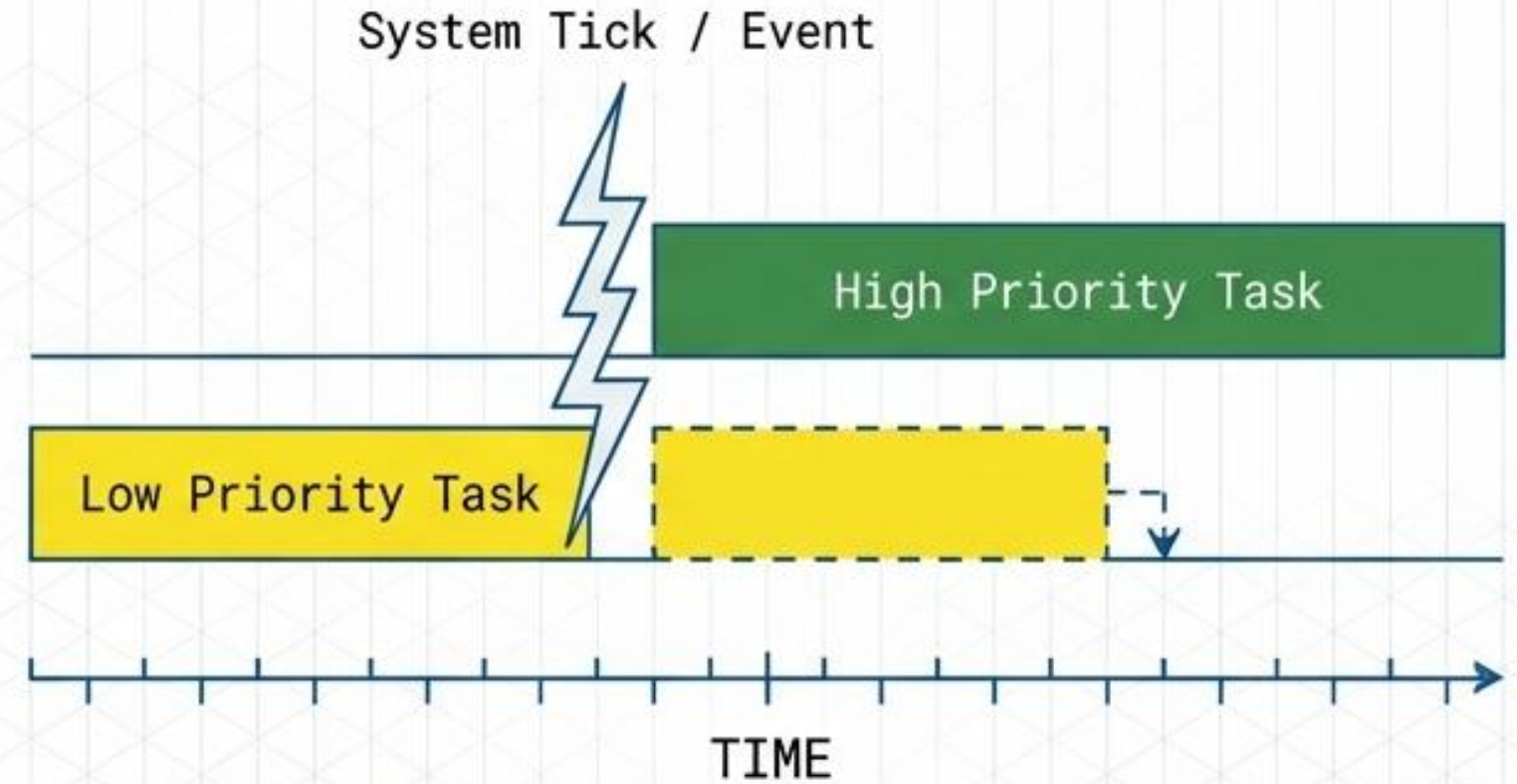
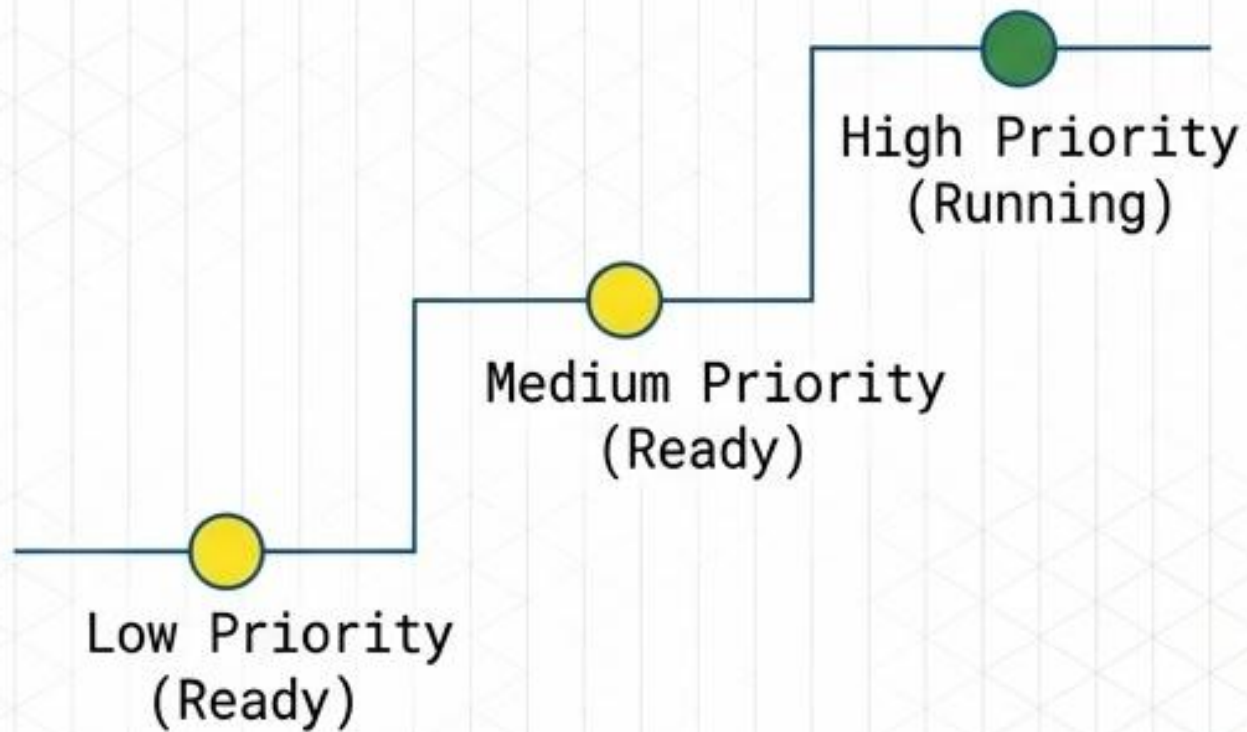
Setiap kali sebuah task dibuat, FreeRTOS mengalokasikan sejumlah ruang dari Heap utama (memori dinamis) untuk didedikasikan secara permanen menjadi Stack pribadi milik task tersebut.

Peringatan Kritis: Perhitungan Stack Size sangat krusial. Jika salah memperkirakan kedalaman pemanggilan fungsi atau variabel lokal, akan terjadi Stack Overflow yang menyebabkan crash sistem.

Siklus Hidup dan State Transisi



Algoritma Scheduler: Preemptive & Priority-Based



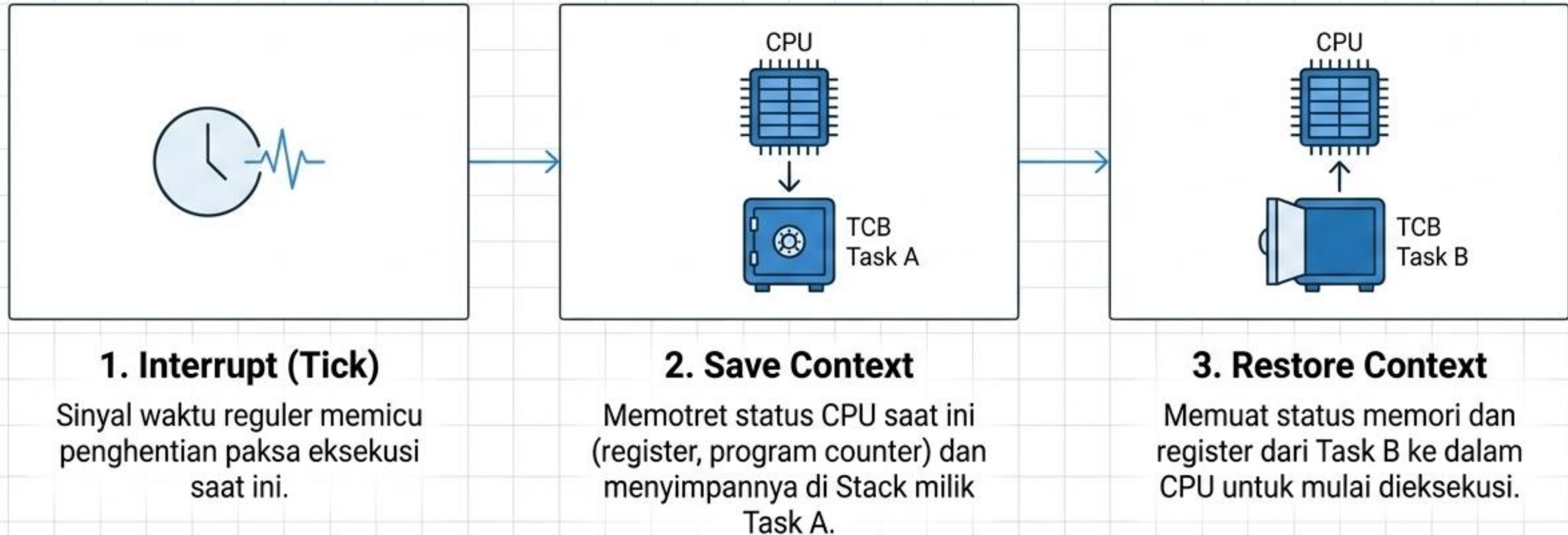
Aturan Absolut CPU

CPU selalu mengeksekusi task berprioritas tertinggi yang saat ini berada dalam state READY. Task prioritas rendah akan dipaksa turun (preempted).

Time Slicing (Round Robin)

Jika terdapat beberapa task pada tingkat prioritas yang sama persis, scheduler akan membagi waktu CPU secara bergantian di antara mereka setiap kali System Tick terjadi.

Context Switching & System Tick



Tick Rate (`configTICK_RATE_HZ`)
Pengatur seberapa sering interrupt ini dievaluasi. Konfigurasi 1000 Hz berarti scheduler melakukan evaluasi status setiap 1 milidetik

Implementasi STM32: Arsitektur Single-Core

Spesifikasi Hardware

Single Core (72MHz),
RAM 20KB.

Integrasi Sistem

Diintegrasikan secara manual
melalui generator kode
STM32CubeMX.

Manajemen Heap

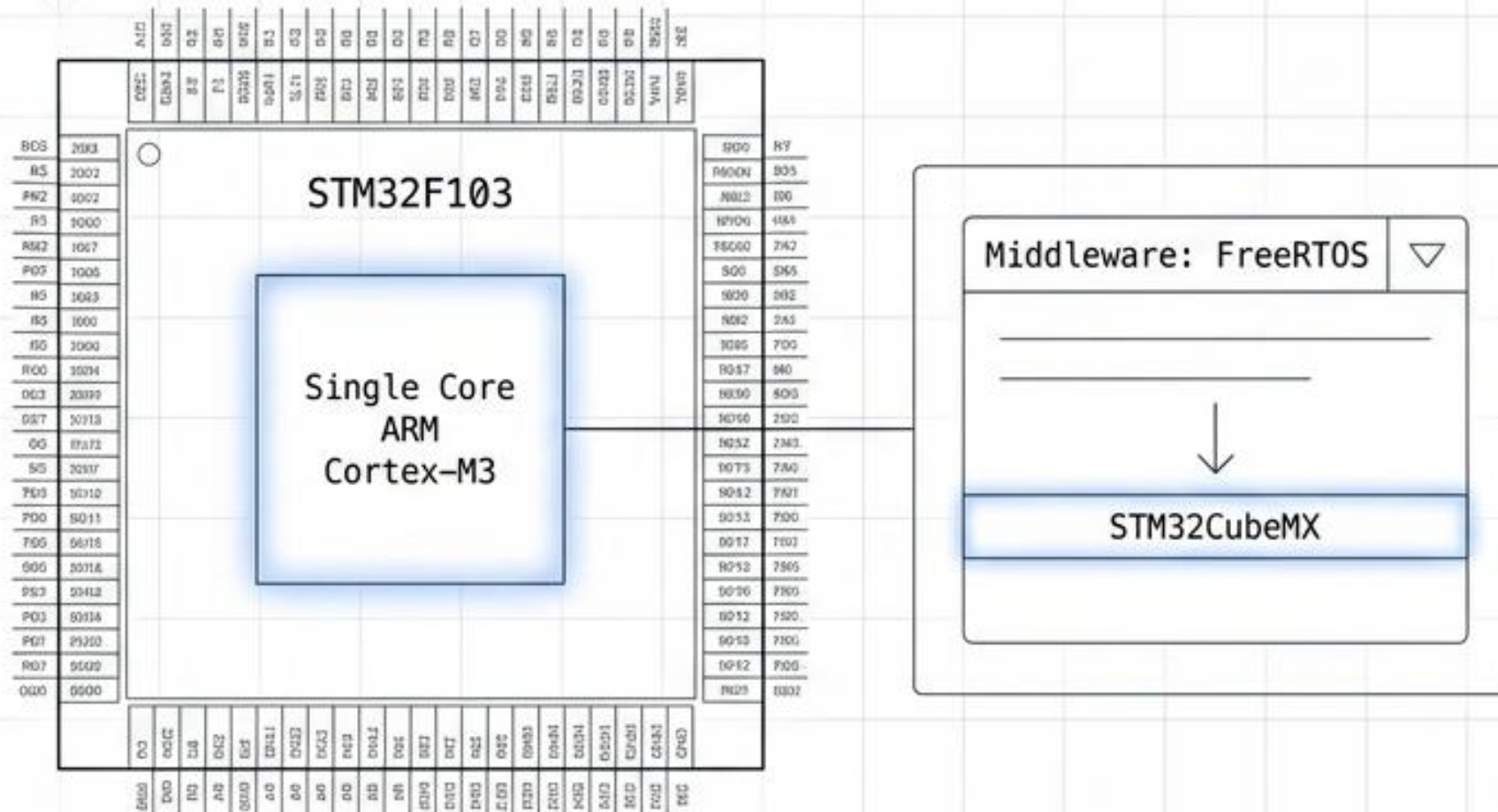
Menggunakan skema heap_4.c secara
default.

Satuan Stack

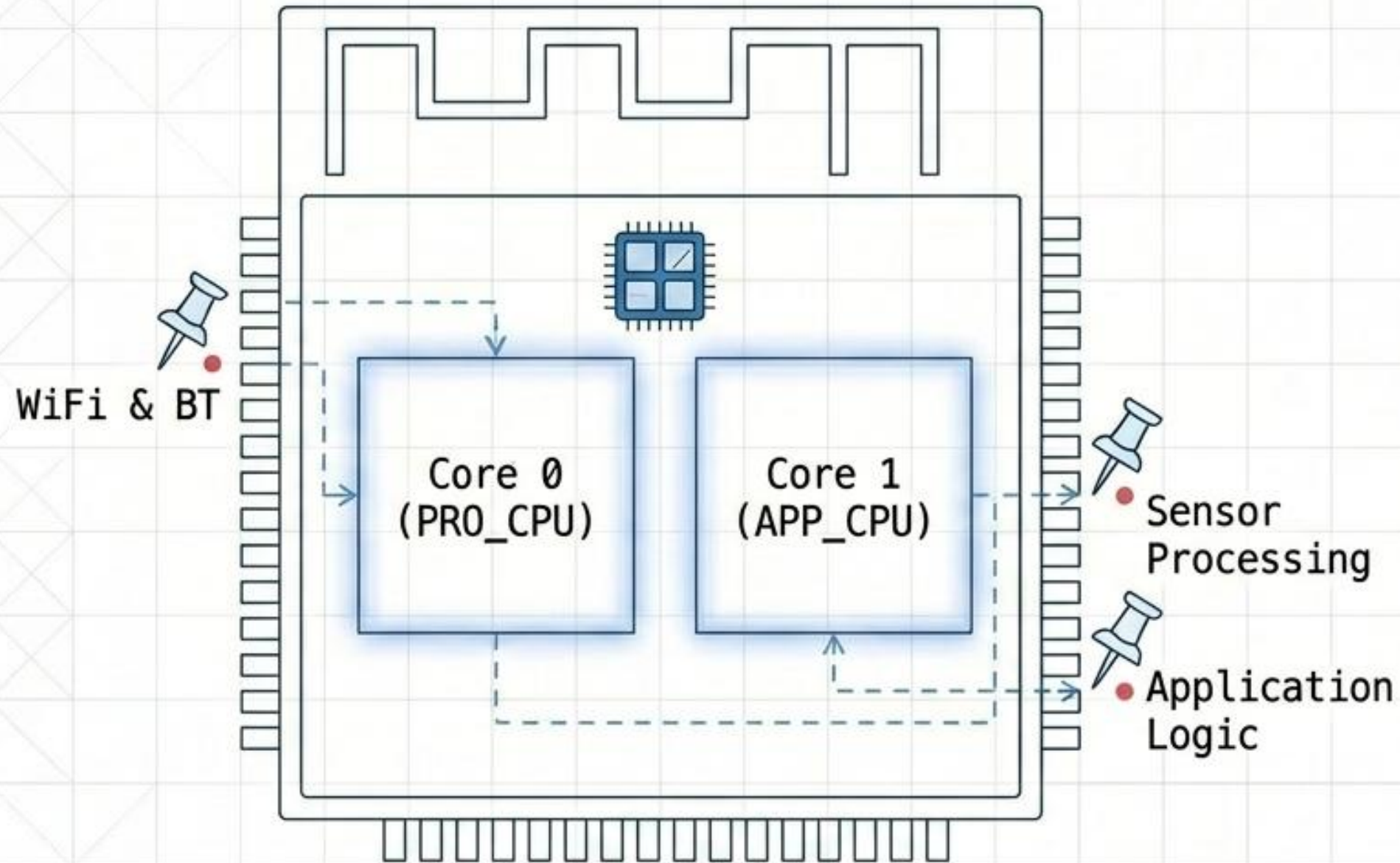
Diukur dalam satuan Words (1 word =
= 4 bytes). Alokasi 100 berarti 400
bytes.

Default Tick Rate

Umumnya diatur agresif pada 1000Hz
(1ms).



Implementasi ESP32: Kekuatan Dual-Core (ESP-IDF)



Spesifikasi Hardware

Dual Core Xtensa (240MHz),
RAM 520KB.

Sistem Built-In

RTOS terintegrasi penuh ke dalam
framework dasar ESP-IDF, bukan
sekadar middleware.

Core Affinity

Pengembang dapat menentukan secara
eksplisit pada core mana task
berjalan (misal: WiFi di Core 0,
Sensor di Core 1).

Satuan Stack

Diukur langsung dalam satuan Bytes
(berbeda drastis dengan STM32).

Default Tick Rate

Lebih efisien daya, diatur pada
100Hz (10ms).

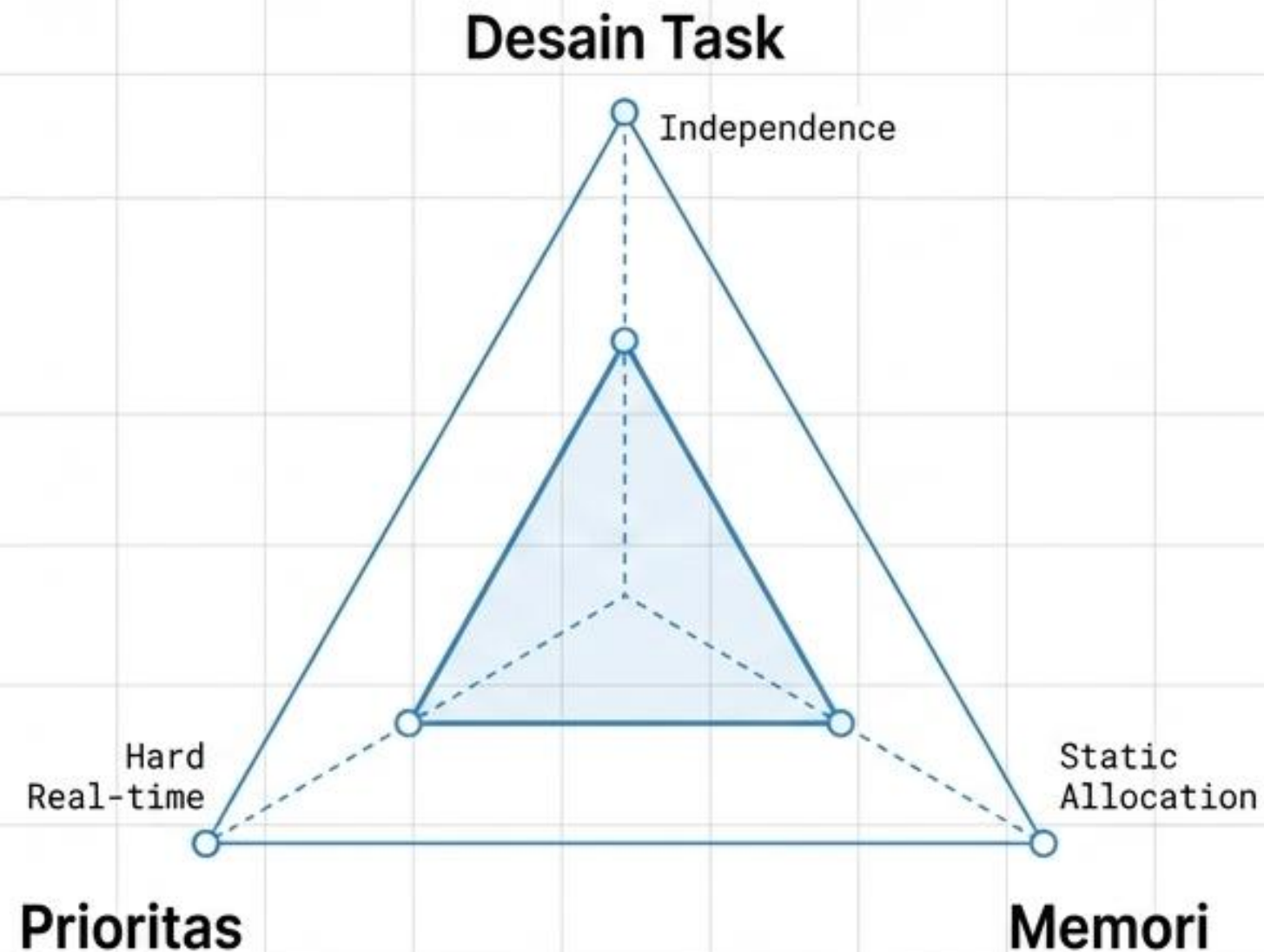
Hardware & RTOS Implementation Matrix

Aspek Metrik	STM32F103	ESP32
Cores	Single (72MHz)	Dual (240MHz each)
Kapasitas RAM	20KB	520KB
FreeRTOS Base	Manual integration	Built-in (Modified)
Satuan Stack	Words (4 bytes)	Bytes
Default Heap	heap_4.c	ESP-IDF heap
Default Tick	1000Hz (1ms)	100Hz (10ms)
Core Affinity	N/A (Single path)	Supported (Pin-to-core)
Watchdog Timer	Konfigurasi Manual	Integrated & Active

Matriks Diagnostik dan Troubleshooting

 Gejala Kritis (Symptom)	 Diagnosis Akar Masalah	 Resep / Solusi
Task tidak berjalan	Prioritas terlalu rendah (Kalah bersaing)	Naikkan tingkat prioritas task.
System terhenti mendadak (Hard Fault)	Stack Overflow (Alokasi ruang terlalu kecil)	Tingkatkan stack size dan gunakan fungsi deteksi overflow.
System Hang / Freeze	Task menguasai CPU terus menerus (tidak yield)	Sisipkan fungsi penunda waktu seperti vTaskDelay.
Perilaku tidak menentu (Erratic)	Priority Inversion (Task rendah mengunci resource)	Gunakan struktur Mutex dengan fitur priority inheritance.
Memory exhausted	Terlalu banyak instance task aktif	Kurangi task dinamis atau perbesar rasio alokasi Heap.

Arsitektur Best Practices: The Deterministic Advantage



1. Desain Task Independen

Pastikan setiap task murni independen. Jangan pernah membuat polling loop kosong; selalu gunakan API blocking berbasis event/timeout agar membebaskan siklus CPU.

2. Hierarki Prioritas Ketat

Alokasikan prioritas tertinggi HANYA pada task dengan latensi **batas keras** (hard real-time / hardware interrupt handler). Rutinitas UI atau Logging harus berada di prioritas terendah.

3. Stabilitas Memori

Hindari pembuatan dan penghapusan task secara dinamis (dynamic creation/deletion) di dalam loop eksekusi cepat untuk mencegah fragmentasi Heap sistem.

Kesimpulan

FreeRTOS memungkinkan ESP32 dan STM32 menjalankan beberapa task secara efisien melalui sistem multitasking. Dengan adanya scheduler, pembagian prioritas, dan mekanisme sinkronisasi, program dapat berjalan lebih terstruktur, responsif, dan sesuai kebutuhan sistem real-time. ESP32 dan STM32 memiliki karakteristik hardware yang berbeda, tetapi konsep penerapan FreeRTOS pada keduanya tetap serupa. Oleh karena itu, FreeRTOS menjadi salah satu solusi penting dalam pengembangan sistem embedded modern, terutama untuk aplikasi IoT, otomasi, dan sistem real-time.